

프롬프트 하나로 데이터 분석 시키기: LangChain으로 배우는 AI 분석

LangChain - Agent, Tool

김종영





User



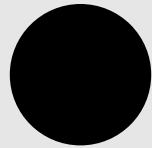
단순 텍스트 생성
vs.
추론/계획 능력



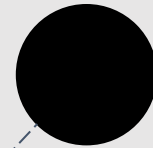
LLM



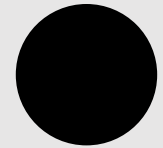
User



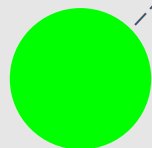
AgentExecutor



Agent



LLM



Tool

Agent - Tool 구조

- LLM은 도구 호출을 할 수 없음
- LLM은 도구 호출을 직접 하면 안됨
- 확장성 및 효율 (도구 때문에 모델 재학습?)



User



단순 텍스트 생성
vs.
추론/계획 능력



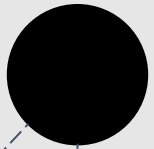
LLM



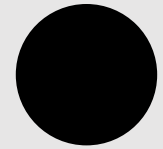
User



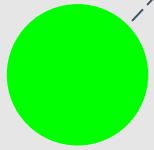
AgentExecutor



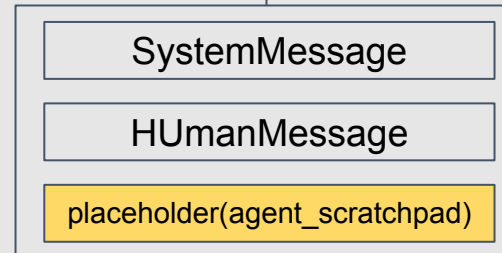
Agent



LLM



Tool



Template

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao^{*1}, Jeffrey Zhao², Dian Yu², Nan Du², Izhak Shafran², Karthik Narasimhan¹, Yuan Cao²

¹Department of Computer Science, Princeton University

²Google Research, Brain team

¹{shunyuy, karthikn}@princeton.edu

²{jeffreyzhao, dianyu, dunan, izhak, yuancan}@google.com

ABSTRACT

While large language models (LLMs) have demonstrated impressive performance across tasks in language understanding and interactive decision making, their abilities for reasoning (e.g. chain-of-thought prompting) and acting (e.g. action plan generation) have primarily been studied as separate topics. In this paper, we explore the use of LLMs to generate both reasoning traces and task-specific actions in an interleaved manner, allowing for greater synergy between the two. Reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while actions allow it to interface with and gather additional information from external sources such as knowledge bases or environments. We apply our approach, named ReAct, to a diverse set of language and decision making tasks and demonstrate its effectiveness over state-of-the-art baselines in addition to improved human interpretability and trustworthiness. Concretely, on question answering (HotpotQA) and fact verification (Fever), ReAct overcomes prevalent

기존 CoT (Chain-of-Thought) 프롬프팅 기법이 언어 모델의 추론 능력 향상에 초점을 맞춘 한계를 극복한 Agent - Tool 아키텍처의 출발이 된 논문

2 REACT: SYNERGIZING REASONING + ACTING

Consider a general setup of an agent interacting with an environment for task solving. At time step t , an agent receives an observation $o_t \in \mathcal{O}$ from the environment and takes an action $a_t \in \mathcal{A}$ following some policy $\pi(a_t|c_t)$, where $c_t = (o_1, a_1, \dots, o_{t-1}, a_{t-1}, o_t)$ is the context to the agent. Learning a policy is challenging when the mapping $c_t \mapsto a_t$ is highly implicit and requires extensive computation. For example, the agent shown in Figure 1(1c) is unable to generate the correct final action (Act 4) to finish the QA task as it requires complex reasoning over the trajectory context (Question, Act 1-3, Obs 1-3). Similarly, the agent shown in Figure 1(2a) fails to comprehend from the context that sinkbasin 1 does not contain peppershaker 1, thus keep producing hallucinating actions.

The idea of ReAct is simple: we augment the agent's action space to $\hat{\mathcal{A}} = \mathcal{A} \cup \mathcal{L}$, where $\mathcal{L}$ is the space of language. An action $\hat{a}_t \in \mathcal{L}$ in the language space, which we will refer to as a *thought* or a *reasoning trace*, does not affect the external environment, thus leading to no observation feedback. Instead, a thought $\hat{a}_t$ aims to compose useful information by reasoning over the current context c_t , and update the context $c_{t+1} = (c_t, \hat{a}_t)$ to support future reasoning or acting. As shown in Figure 1, there could be various types of useful thoughts, e.g. decomposing task goals and create action plans (2b, Act 1; 1d, Thought 1), injecting commonsense knowledge relevant to task solving (2b, Act 1), extracting important parts from observations (1d, Thought2, 4), track progress and transit action plans (2b, Act 8), handle exceptions and adjust action plans (1d, Thought 3), and so on.

However, as the language space $\mathcal{L}$ is unlimited, learning in this augmented action space is difficult and requires strong language priors. In this paper, we mainly focus on the setup where a frozen large language model, PaLM-540B (Chowdhery et al., 2022)¹, is prompted with few-shot in-context examples to generate both domain-specific actions and free-form language thoughts for task solving (Figure 1 (1d), (2b)). Each in-context example is a human trajectory of actions, thoughts, and environment observations to solve a task instance (see Appendix C). For the tasks where reasoning is of primary importance (Figure 1(1)), we alternate the generation of thoughts and actions so that the task-solving trajectory consists of multiple thought-action-observation steps. In contrast, for decision making tasks that potentially involve a large number of actions (Figure 1(2)), thoughts only need to

¹We show some GPT-3 (Brown et al., 2020) results in Appendix A.1, which outperforms PaLM-540B.

=====GENERAL SETUP=====

- 에이전트가 외부 환경과 인터랙션해서 만들어지는 정보
 - 에이전트의 행동 a , 행동으로 얻은 결과 o
 - c 는 지금까지의 a 와 o 를 모아 놓은 것
 - 시점 t 를 적용; a_t, o_t, c_t
- 제일 처음에는 o_1 만 있음 $\rightarrow$ 에이전트가 아무것도 안해도
 - $c_1 = (o_1)$
 - $c_2 = (o_1, a_1, o_2)$
 $= (c_1, a_1, o_2)$
 - $c_3 = (o_1, a_1, o_2, a_2, o_3)$
 $= (c_2, a_2, o_3)$
 - ...
 - $c_t = (c_(t-1), a_(t-1), o_t)$
- ① $c_(t-1) \rightarrow a_(t-1)$
- ② $a_(t-1) \rightarrow o_t$
- ReAct는 "텍스트를 만들어내는 것도 action이다"에서 출발
 - $\hat{\mathcal{A}} = \mathcal{A} \cup \mathcal{L}$
 - 여기에서 $\mathcal{A}$ 는 도구 호출과 같은 전통적 액션
 - $\mathcal{L}$ 은 언어 모델이 텍스트를 만들어 내는 "새로운 액션"
- Thought $\rightarrow$ Action $\rightarrow$ Observation에서
- Thought $\rightarrow$ Action는 언어 모델이
- Action $\rightarrow$ Observation는 도구 호출
- c 는 지금까지의 (Thought, Action, Observation)
- ① $c_(t-1) \rightarrow a_(t-1)$ 는 확률 분포였지만 이제는 LLM
- (Thought $\rightarrow$ Action $\rightarrow$ Observation) + Thought $\rightarrow$ Action $\rightarrow$ Observation의 형태로 c 를 사용
- o_1 은 초기 프롬프트에 해당,